SQL JOINs

Visual Guide - Row by Row

See how SQL reads two tables and puts data together.

Color key:

Color	Meaning
Yellow row	SQL is reading this row now
Green row	Match found between the two tables
Red row	No match - row is removed

Our two tables:

EMPLOYEES (LEFT table - comes after FROM)

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

DEPARTMENTS (RIGHT table - comes after JOIN)

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

Both tables have a column called dept_id. This is the bridge. SQL uses this column to connect the two tables.

Part 1: What Problem Does a JOIN Solve?

Look at the EMPLOYEES table. You can see Steven, Neena, Lex, Diana, Kimberely. You can see their dept_id numbers (90, 10, NULL...). But you can NOT see the department name. The department name is in the DEPARTMENTS table. So you have a problem: the information you need is in TWO different tables.

A JOIN solves this problem. It takes data from two tables and puts it together in one result. It connects them using the bridge column (dept_id). Think of it like this: you have a list of people with a team number. You have another list of teams with their team number. JOIN matches the team numbers and shows: person name + team name together.

Part 2: Understand the Query (Line by Line)

Before we look at the rows, let us understand each line of the query.

```
SELECT e.first_name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

Line 1: FROM employees e

This tells SQL: start with the EMPLOYEES table. Call it e for short. The letter e is a short name (alias). After this, every time you write e.first_name, SQL knows you mean the first_name column in the employees table. You do not have to write the full table name every time.

Line 2: INNER JOIN departments d

This tells SQL: also bring the DEPARTMENTS table. Call it d for short. The word INNER JOIN is the type of JOIN. You can change this word later to LEFT JOIN, RIGHT JOIN, or FULL OUTER JOIN. Each type works a little different. We will see that soon.

Line 3: ON e.dept_id = d.dept_id

This is the matching rule. This is the most important line. It tells SQL: connect rows where the dept_id in employees (e.dept_id) is the same as the dept_id in departments (d.dept_id). Without this line, SQL can not connect anything.

Line 4: SELECT e.first_name, d.dept_name

This tells SQL: in the result, show only two columns. first_name from the employees table (e.first_name) and dept_name from the departments table (d.dept_name). The result does NOT show all columns. It shows ONLY the columns you write after SELECT. If you write e.first_name, e.last_name, d.dept_name then you see three columns.

Why do we write e. and d. before column names?
Because both tables have a column called dept_id. If you just write dept_id, SQL does not know: which one? The one in employees or the one in departments? The letter before the dot tells SQL: this column is from THAT table. e.dept_id = dept_id from employees. d.dept_id = dept_id from departments.

Part 3: What is NULL? (Important!)

NULL means empty. It means there is no value. It is not zero. It is not a space. It is nothing. Like an empty box.

Think of it like this: Lex has a dept_id. But his dept_id is NULL. That means Lex has NO department. He is not in any team.

> Very important rule: NULL can not match anything. Even NULL = NULL is NOT true. Think of it like this: if I ask you "does box A equal box B" and both boxes are empty, you still can not say they are the same empty thing. They are both empty, but SQL says: I can not compare two things that do not exist. So when Lex has NULL for dept_id, SQL can not find a match for him in the departments table. That is why Lex is removed in INNER JOIN.

Part 4: INNER JOIN - Row by Row

INNER JOIN = keep only rows that match in BOTH tables. If a row has no match, it is removed. Only matching rows survive.

```
SELECT e.first_name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

SQL starts with the LEFT table (employees) and reads it row by row, from top to bottom. For each row, it takes the dept_id value and looks for the same value in the RIGHT table (departments).

Step 1: SQL reads row 1 from employees

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

Steven's dept_id is 90. SQL looks in departments. Is there a row with dept_id = 90?

YES! Row 3 in departments has dept_id = 90 (Executive). MATCH!

Now SQL reads the SELECT columns from both tables:
From employees (e): first_name = "Steven"
From departments (d): dept_name = "Executive"
Result row: Steven | Executive

Step 2: SQL reads row 2 from employees

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

Neena's dept_id is 90. SQL looks in departments. Is there a row with dept_id = 90?

YES! Same row: dept_id = 90 (Executive). MATCH!

Result row: Neena | Executive

Step 3: SQL reads row 3 from employees

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

Lex's dept_id is NULL. SQL looks in departments. Is there a row with dept_id = NULL?

NO! NULL can not match anything. NO MATCH.

Lex is REMOVED from the result. He is gone.

Why? Because this is INNER JOIN. INNER JOIN removes rows with no match. Remember from Part 3: NULL can not match any value in the departments table.

Step 4: SQL reads row 4 from employees

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

Diana's dept_id is 10. SQL looks in departments. Is there a row with dept_id = 10?

YES! Row 1 in departments has dept_id = 10 (Administration). MATCH!

Result row: Diana | Administration

Step 5: SQL reads row 5 from employees

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

Kimberely's dept_id is 10. SQL looks in departments. Is there a row with dept_id = 10?

YES! Row 1 in departments has dept_id = 10 (Administration). MATCH!

Result row: Kimberely | Administration

What about departments with no employees?

SQL finished reading all 5 rows from employees. But look at the departments table. Marketing (dept_id = 20) has NO employee with dept_id = 20. Accounting (dept_id = 110) has NO employee with dept_id = 110. In INNER JOIN, these departments are also removed. INNER JOIN removes unmatched rows from BOTH tables.

INNER JOIN Final Result:

first_name	dept_name
Steven	Executive
Neena	Executive
Diana	Administration
Kimberely	Administration

We started with 5 employees + 4 departments. Result has only 4 rows.
Lex is gone (NULL dept_id, no match). Marketing is gone (no employee has dept_id 20). Accounting is gone (no employee has dept_id 110). INNER JOIN keeps ONLY matching rows.

Part 5: LEFT JOIN

LEFT JOIN = keep ALL rows from the LEFT table (employees). If a row has no match, do NOT remove it. Just write NULL for the right table columns.

```
SELECT e.first_name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

Steps 1, 2, 4, 5 are the same as INNER JOIN (they all have a match). Only Step 3 is different. Let us look at that step again.

Step 3 with LEFT JOIN (Lex):

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

Lex's dept_id is NULL. No match in departments.

But LEFT JOIN does NOT remove Lex! He stays.
SQL writes NULL for dept_name because there is no matching department.
Result row: Lex | NULL

LEFT JOIN Final Result:

first_name	dept_name
Steven	Executive
Neena	Executive
Lex	NULL
Diana	Administration
Kimberely	Administration

5 rows now! Lex is back. His dept_name is NULL. Marketing and Accounting are still gone (they are in the RIGHT table). LEFT JOIN protects the LEFT table only.

Part 6: RIGHT JOIN

RIGHT JOIN = keep ALL rows from the RIGHT table (departments). This time, the reading direction changes. SQL starts from the RIGHT table.

```
SELECT e.first_name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

Important: Now SQL reads from departments (RIGHT table), row by row. For each department, it looks for an employee with the same dept_id in the employees table.

Administration (dept_id = 10): Diana and Kimberely match. Both are kept.
Executive (dept_id = 90): Steven and Neena match. Both are kept.

Now look at what happens with Marketing:

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

Marketing's dept_id is 20. SQL looks in employees. Is there any employee with dept_id = 20?

NO! No employee has dept_id = 20. NO MATCH.
But RIGHT JOIN does NOT remove Marketing! It stays.
SQL writes NULL for first_name because there is no matching employee.
Result row: NULL | Marketing

(The same happens for Accounting, dept_id = 110)

RIGHT JOIN Final Result:

first_name	dept_name
Diana	Administration
Kimberely	Administration
NULL	Marketing
Steven	Executive
Neena	Executive
NULL	Accounting

6 rows! Marketing and Accounting are back with NULL. But Lex is gone (he is in the LEFT table, RIGHT JOIN protects the RIGHT table).

Part 7: FULL OUTER JOIN

FULL OUTER JOIN = keep ALL rows from BOTH tables. Nothing is removed. No match = write NULL on the empty side.

```
SELECT e.first_name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d
ON e.dept_id = d.dept_id;
```

This is the same as LEFT JOIN + RIGHT JOIN together. All matching rows are kept. Lex is kept (from LEFT side). Marketing and Accounting are kept (from RIGHT side).

FULL OUTER JOIN Final Result:

first_name	dept_name
Steven	Executive
Neena	Executive
Lex	NULL
Diana	Administration
Kimberely	Administration
NULL	Marketing
NULL	Accounting

7 rows! Everyone is here. No row is lost. Lex, Marketing, and Accounting have NULL where there is no match.

Part 8: All 4 JOINs Compared

Row	INNER	LEFT	RIGHT	FULL OUTER
Steven | Executive	YES	YES	YES	YES
Neena | Executive	YES	YES	YES	YES
Lex | NULL	NO	YES	NO	YES
Diana | Admin	YES	YES	YES	YES
Kimberely | Admin	YES	YES	YES	YES
NULL | Marketing	NO	NO	YES	YES
NULL | Accounting	NO	NO	YES	YES
Total rows	4	5	6	7

Simple rules:

JOIN Type	What it does (simple words)
INNER JOIN	Only matching rows from both tables. No match = removed.
LEFT JOIN	ALL rows from left table (FROM). Right table: only matches.
RIGHT JOIN	ALL rows from right table (JOIN). Left table: only matches.
FULL OUTER JOIN	ALL rows from BOTH tables. Nothing is removed.

How to remember:

LEFT = the first table (FROM). Keep all its rows.
RIGHT = the second table (JOIN). Keep all its rows.
INNER = keep nobody extra. Only matches.
FULL = keep everything. Lose nothing.

When to use which:

You want ...	Use ...
Only employees who have a department	INNER JOIN
ALL employees, even without department	LEFT JOIN
ALL departments, even without employees	RIGHT JOIN
ALL employees AND ALL departments	FULL OUTER JOIN
Find employees with no department	LEFT JOIN then WHERE dept_id IS NULL

Common mistakes:

Mistake 1: Forgetting ON. ON is the matching rule. Without it, SQL can not connect anything.
Mistake 2: Writing just first_name without e. when you have a JOIN. SQL does not know which table to use. Always use e.first_name or d.dept_name.
Mistake 3: Using WHERE instead of ON for the match. ON is for connecting tables. WHERE is for filtering after the join.